Malware Forensics in Computers

1st Edward Ettesvoll
Norwegian University of Science and Technology
Gjøvik, Norway
edwarde@stud.ntnu.no

2nd Fariss Hadi
Norwegian University of Science and Technology
Gjøvik, Norway
farissh@stud.ntnu.no

3rd Martynas Palumickas
Norwegian University of Science and Technology
Gjøvik, Norway
martynap@stud.ntnu.no

4th Tryggvi T. Zabelberg
Norwegian University of Science and Technology
Gjøvik, Norway
tryggviz@stud.ntnu.no

***Abstract*—In this project, we set out to learn more about the behaviors and characteristics of malicious software by taking a practical approach to malware analysis. Our goal with the project is to perform static and dynamic analysis of malware samples in a secure, controlled, and repeatable approach. In order to meet this goal we set up manual and automated sandbox environments for analysis, where we used forensics tools and techniques to examine changes on different computer systems.**

***Index Terms*—Malware, Analysis, Static, Dynamic, VirtualBox, Trojan, Virus, Network, Forensics, Cuckoo**

I. Introduction

Malware analysis serves multiple important purposes and is an essential component of cybersecurity. It does this by recognizing and comprehending new malware, which helps to detect and prevent threats. Malware analysis helps with effective incident response in the event of a security incident by offering insights into the behavior of malware and the degree of compromise. It also aids in the attribution and cyber threat intelligence, which helps create a thorough grasp of threat actors and their strategies. By locating emerging attack vectors and vulnerabilities, malware analysis aids in security research and development and facilitates the development of intrusion detection and antivirus signatures. Comprehending the behavior of malware is crucial for formulating countermeasures and security protocols. Furthermore, it is essential to forensic investigations as it helps with evidence collection and event reconstruction.

We have taken a more hands-on approach to our malware analysis topic, requiring the use of a variety of tools, conducting both static and dynamic analysis, and learning from experience. The majority of the work is practical rather than theoretical. Naturally, in order to analyze malicious malware, we will need to set up a safe sandbox environment so as not to damage our machines. We will provide a description of our environment setup where we will aim to be as clear as possible while avoiding overly technical terminology. In response to feedback, we decided that analyzing many malware types would be overly ambitious, therefore we decided to examine three samples to keep it simple. We encountered a number of problems when attempting to set up the environments for the project. We, therefore, decided to add a troubleshooting section to cover the problems we encountered with the intention of providing some pointers to anyone who would want to configure and set up the same environments as we did in this project.

A. Structure

After the introduction, we start with the knowledge and theory chapter where the definitions for all the terms and concepts we used for this project are, so that the reader can get a better understanding of the contents of the project. Following the theory and knowledge chapter, there are the motivation, scope, and methodology sections which discuss why and how this project was done. Chapter four focuses on the problems that we encountered and how we solved them, we agreed to include this chapter since it took up a significant amount of time during work on the project. Further on we explain the environments we used for the analysis and how it was set up. The next chapter focuses on the main objective of the project which is the analysis of the samples. Lastly, we conclude our project with a short summary and some of the results.

II. Knowledge and Theory

A. Virtual Machine (VM)

Multiple operating systems can run simultaneously on a single host computer thanks to a virtual machine (VM), a software-based emulation of a physical computer. The virtual machine (VM), which is overseen by a hypervisor, is equipped with virtualized CPU, memory, storage, and network interfaces, along with an autonomous guest operating system. With the help of virtual machine (VM) features like snapshotting and cloning, users can efficiently test and deploy their applications by capturing and duplicating particular states. Because they are separate from one another, problems in one VM won't affect the others [1].

A virtual machine (VM) is an essential tool in malware analysis because it provides a safe, isolated environment for the safe execution and inspection of potentially malicious code. The host system and network of the analyst are protected

by this isolation, which guarantees that the malware's effects are confined to the virtualized environment. Before running malware, analysts can take a snapshot of a virtual machine (VM) and use it to analyze the impact of the malicious code and roll it back. Virtual machines (VMs) have the capability to support dynamic analysis, which makes it possible to observe the behavior of malware while it is being executed and thus aid in comprehending its functionality. Virtual machines (VMs) are also useful for network traffic monitoring, providing information about possible threats and communication patterns.

B. Cuckoo Sandbox

Cuckoo Sandbox is an open-source automated dynamic malware analysis system. Its purpose is to examine malicious files and URLs in order to determine how they behave and what effects they might have on a system. With a successful deployment of Cuckoo, it is similar to running a combination of static and dynamic analysis tools, in one universal package. Therefore, Cuckoo's main objective is to give security researchers and analysts a highly automated platform to study and analyze malware's internal operations in a safe and private setting. It has a user-friendly GUI (Graphical User Interface) with different tabs and windows for every section [2].

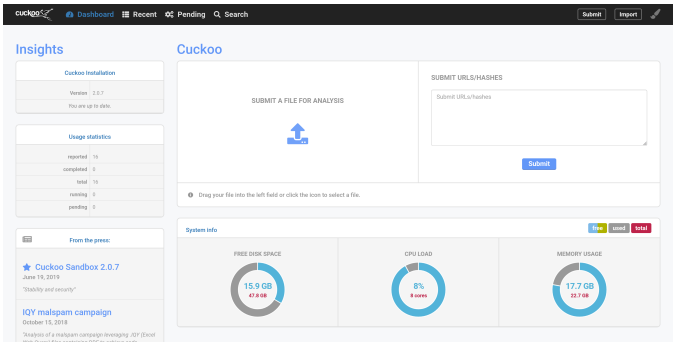


Fig. 1. Cuckoo User Interface

C. Static Malware Analysis

A technique for analyzing malicious software (malware) without running it is called static malware analysis. In this kind of analysis, the traits and attributes of a malware sample are examined through an examination of its file attributes, code and structure, and other static features. The benefit of static malware analysis is that it lowers the possibility of accidental infection or damage by enabling security researchers and analysts to evaluate a malware sample quickly and without running it. It might not, however, record the entire spectrum of dynamic behaviors displayed by the malware while it is being executed. Thus, in order to give a more thorough understanding of the threat, a thorough malware analysis frequently incorporates both static and dynamic analysis techniques [3].

Attributes as simple as the file type or the icon of the file can be an indicator of malware. A good example of this is executable .exe files that are disguised as PDF files. Changing the icon of an executable file to an icon resembling a PDF is a simple task that does not require any advanced computer or programming knowledge. Changing the file icon to a Windows logo can make the file appear to resemble a Windows update file. Tools such as PEStudio are often used in static malware analysis to extract static data from files without running them.

Malware will often use packing to obfuscate their behavior. This can make the static analysis more difficult if the packer used to pack the malware is unknown. Often however, common depacking tools such as Exeinfo PE and Packer can be used to identify this. In the event that no depacking tool is able to reveal whether the malware is packed or not, the level of entropy (difference between the malware's size in memory and on disk) can give a good indication.

D. Dynamic Malware Analysis

The information security company VMRay defines dynamic malware analysis as "executing or "detonating" suspicious programs within a virtual sandbox environment and closely monitoring their behavior. (...) allows security analysts to safely analyze potentially harmful malware without putting the target system at risk" [4].

Our objective with using dynamic malware analysis is to inspect and observe live malware sample behavior in a safe, controlled, and repeatable manner.

Two samples, one benign and one malicious can appear incredibly similar to each other before execution.

Two samples both using the Win32 function GetKeyState [5], could use the function in vastly different ways. GetKeyState is a function used by applications on Windows to retrieve the keyboard state on a host. This function can be used by games to retrieve keyboard inputs which then would be used by the player to control the game. At the same time, this function could used by a malicious keylogger program in order to retrieve the user's keyboard input [6]. This input could be used by the malicious keylogger program to steal the user's password or sensitive information. This is where dynamic malware analysis comes into play. Two samples can appear similar before they are executed, with both of the samples having the functionality to access the user's keyboard inputs for instance. How this functionality is used, whether legitimately or maliciously can in some cases only be determined by executing them, looking at their behavior.

E. Malware

The term malware refers to a broad and constantly changing class of destructive programs created with the express purpose of interfering with, disrupting, or taking advantage of computer systems, networks, and users. These malicious organizations often have evil intentions when they operate, which can include anything from system manipulation and unauthorized access to data theft and financial extortion. It encompasses a wide range of malicious software types, each serving different purposes [7].

- Viruses: Infect executable files, replicating when the infected file is executed.

- Worms: Self-replicating programs that spread across networks without human intervention, exploiting vulnerabilities.
- Trojans (Most Popular): Disguise themselves as legitimate software, opening a backdoor for unauthorized access.
- Ransomware: Encrypts files and demands payments for their decryption, designed to extort money from individuals or organizations.
- Spyware: Gather information about users or organizations without their knowledge, including keystrokes and browsing habits.
- Adware: Display unwanted advertisements, potentially impacting the user experience.
- Rootkits: Provides hidden, unauthorized access to a computer or network, often compromising the operating system.
- Botnets: Consists of a network of computers (bots) controlled by a central entity, often used for malicious purposes.

F. IOCs (Indicators of Compromise)

In the field of cybersecurity, indicators of compromise, or IOCs, are essential components that function as observable indicators of possible malicious activity occurring within a computer system or network. These indicators are crucial for quick threat detection and response because they provide concrete proof of compromise and can take the form of file hashes, IP addresses, domain names, and URLs. Cybersecurity experts can learn more about threat actors' tactics, techniques, and procedures by examining and keeping an eye on these indicators [8].

IOCs play a vital role in the continuous fight against cyber threats, going beyond simple identification. IOCs help cybersecurity professionals better understand attack patterns, which helps them strengthen defenses, create preventative measures, and share the threat intelligence with the community. By pooling IOCs in this way, the cybersecurity community can respond to new threats and strengthen digital environments against compromise, creating a collaborative and adaptable cybersecurity landscape. The strategic use of IOCs continues to be essential for protecting digital assets and guaranteeing the resilience of information systems in the face of ever-evolving cyber threats.

G. Command & Control (C2) Communication

The communication channel created between a compromised system (such as a computer infected with malware) and a remote entity exercising control over it is referred to in cybersecurity as command and control (C2) communication. Usually, a malevolent actor or an attacker controls this remote entity [9].

C2 communication is primarily used to enable commands to be sent to and information to be received from the compromised system by the attacker. These instructions might tell you to carry out particular tasks, steal information, or update the malware. The compromised system, sometimes called a zombie or bot, is a component of a larger compromised system network called a botnet.

III. Motivation, Scope, and Methodology

A. Motivation

Malware is something we encounter often, especially in cybersecurity. Having the knowledge and know-how of the software and tools that are used to analyze the characteristics of malware is therefore particularly useful. For some of us who plan to take the forensics track, this is even more relevant. Moreover, it generally piques our interest since we mostly remove malware as soon as it's detected by our antivirus. Taking the opportunity to look further into the topic of malware analysis and see more than surface-level details is therefore interesting.

B. Scope

The group will be split into two pairs and each pair will analyze a malware sample. In addition, the group will analyze a third sample using the Cuckoo Sandbox tool. To avoid compromising or damaging our machines the analysis will be performed in a safe and controlled Virtual Machine (VM) environment. In the beginning, it was planned to use Cuckoo Sandbox more extensively, but due to technical difficulties, Cuckoo will be used on only one sample.

C. Methodology

For this project, we decided to select three malware samples without any preliminary knowledge about the samples and perform static and dynamic analyses of them in order to assert their functionality and behavior. The reasoning behind us not wanting to have preliminary knowledge of the samples is for us to be able to use procedures of doing analysis and analysis techniques without being led to conclusions. If we had a lot of prior knowledge of the samples, for example knowing that the sample is ransomware, we would look for behaviors and functionality specifically used by ransomware. In a real-world scenario, it is expected to use all the information you can get about the sample before doing the analysis. However, we felt that the learning outcome would be improved by not using any of this information for our analysis.

The analysis of the first and second malware samples will be performed in a safe isolated sandbox consisting of two side-by-side virtual machines on a single host. The virtual machines will be configured in a way where only they can communicate with each other, with one of the virtual machines acting as a sandbox/testing environment and the other virtual machine emulating a gateway out to the Internet collecting information from the sandbox/testing environment.

The group will use a variation of different tools in order to perform the analysis, with the specific tool depending on the kind of analysis being done and the preference of the individual group member doing the analysis.

The antivirus will be disabled during the analysis in order to see what the samples are capable of without disruption.

Having the antivirus enabled prevents us from transferring the samples to the sandbox environment since they get removed instantly.

The analysis of the third malware sample will be performed with Cuckoo Sandbox. Cuckoo will be installed directly on a host Ubuntu system and VM(s) used for running the malware will be deployed on top of this host system. Note that we will primarily deploy a single VM, but it is possible (and very intuitive) to deploy multiple VMs at once, in the event that, for instance, per-analysis network routing is desirable [10].

All of the samples were downloaded from the malware sample exchange website MalwareBazaar [11][12][13].

The malware samples we performed analysis on for this project are listed below in tables displaying their associated attributes.

Malware Sample 1:

File name	1855c5782eaa8b535345d71d18ff3a9f.exe
File hash (SHA256)	e92de9eadeef273bd294c6eceb92f7507 68766a79c215843e948f37b95bb6723
File size	0.535MB
File type	EXE
Downloads	320
First seen	2023-11-13 17:12:30 UTC

TABLE I
MALWARE SAMPLE 1

Malware Sample 2:

File name	0c5b40fbabd22d7b3717286e6e4df432.exe
File hash (SHA256)	7e390ca86f3a591a740e6aed05214cf7 5773e0d38dd70fd194fe26f12e876123
File size	1.26MB
File type	EXE
Downloads	283
First seen	2023-11-11 17:40:23 UTC

TABLE II
MALWARE SAMPLE 2

Malware Sample 3:

File name	12f4b96090bb7a1aa45768f68870ef90.exe
File hash (SHA256)	ff63976d5cc3f2965ef707e4073440ca 0fd9d976a02ccc766bf7a4d9af9144e9
File size	53.3MB
File type	EXE
Downloads	684
First seen	2023-11-07 23:05:17 UTC

TABLE III
MALWARE SAMPLE 3

IV. TROUBLESHOOTING

During the setup of the malware analysis environment, the group ran into several problems. Most of these were related to the setup of Cuckoo Sandbox, where the group decided not to install Ubuntu directly on their host system (e.g. as a dual-boot) and instead in a VM, as part of the Cuckoo setup. This was necessary for various reasons, including laptops at service and a lack of storage space on laptops used by the members.

As a result, the group struggled to install the Windows virtual machine inside the Ubuntu virtual machine. Preliminary setup was successful, including setting up Cuckoo itself on the Ubuntu VM, but when initializing the Windows VM through VMCloak, the group found themselves stuck on a non-responding system.

It should be noted that a lot of the challenges experienced by the group were a consequence of a complicated setup process, caused in part by the lack of updates provided by the Cuckoo developers in recent years. The last time Cuckoo was officially updated was at the time of writing as far back as June 2019 [14]. This led to the group failing very early in the process upon initial setup attempts on their existing Debian-based VMs (Ubuntu/Kali), because of older Cuckoo-required versions of Python and other packages not installing correctly. This caused the need to dig up older OS versions that were fully supported by Cuckoo (Ubuntu 18.04). After trying these better-supported OS versions, the group would come further but generally fail on the setup of the Windows VM. Having all these issues and getting nowhere, some members decided to drop the Cuckoo Sandbox setup and go for a different setup (Windows VM and REMnux VM).

The group ended up completing a setup of Cuckoo by using a host Ubuntu system, after a long process of trial and error. In retrospect, we speculate that the primary cause behind not being able to install and setup a VM within a VM was related to using a VM instead of a host Ubuntu machine. A solution here would be to install Ubuntu directly on one's system. Note that, due to lack of updates from the Cuckoo developers in recent years and subsequently out-of-date dependencies (mostly Python-related), we ran into troubles installing Cuckoo on the newest LTS release of Ubuntu, version 22.04. Instead, it is highly recommended to use either Ubuntu 18.04 or 20.04, where we used the former.

A. Bypassing required Microsoft sign-in on Windows 11

While setting up the Windows 11 virtual machine, we ran into an issue where we had to sign in using a Microsoft account in order to complete the installation of the operating system. The group wanted to avoid using a Microsoft account, due to us not wanting to have our accounts signed in on a virtual machine that was going to be used to test malware samples. To achieve this, it was necessary to do a little research, since Microsoft has made it more difficult with this version of Windows. We followed what was outlined in the articles [15] and [16], which in summary revolves around pressing a specific key combination to open up the Command Prompt during the early stages of the setup, writing a few commands to disable the Internet connection, and in turn being allowed to continue with only a local account, because of a lack of network connection.

V. Environment/Setup

A. Hybrid Analysis Sandbox Environment

The following section describes the setup and configuration of the sandbox environment we used for our hybrid analysis of malware samples 1 and 2 in the project. Hybrid analysis is the combined use of static and dynamic malware analysis techniques [17].

For the virtual machines, we either used the VirtualBox or VMWare hypervisors.

Configuration of Virtual Machines:

- Network configuration in VirtualBox/VMWare
  - Created a host-only virtual network that was assigned to both of the Virtual Machines. The virtual machines can communicate with each other, but cannot communicate with the host machine. The VMs will not be able to communicate to the Internet as well with this setup.
- Windows 10 or 11 Virtual Machines
- REMnux Ubuntu-based Virtual Machine

Malware samples are executed on the Windows virtual machine (VM), which sends data to the REMnux VM. The REMnux VM simulates the Internet, whereas the Windows VM interprets this as the ”real” Internet. We configured the Windows and REMnux virtual machines using the tool INetSim. Using this configuration, the Windows virtual machine connects to the REMnux virtual machine every time it needs access to the Internet. The REMnux VM can be thought of as the gateway to the Internet for the Windows VM. Using this method, we can observe the network behaviors of the programs we are doing analysis on.

We followed the guide provided in ”Inetsim/Virtual Lab installation and configuration guide” [18] created by the University of Maryland and the ”Analysis Platform” guide from the NTNU course ”Reverse Engineering and Malware Analysis” in order to set up and configure the environment [19].

We configured our hypervisors, this being VirtualBox and VMWare to be more secure. This was done by disabling clipboard- and file sharing between the virtual machines and the host machine. We did this to limit the access a malicious sample could have to our host machines.

Another setting we changed was on the Windows virtual machine. For this VM we decided to disable the default antivirus Windows Defender. The reason for us disabling the antivirus is that we wanted to observe how the sample would perform without any interference. Having the antivirus enabled also makes it more difficult to transfer the malware sample to the virtual machine.

B. Automated Cuckoo Sandbox Environment

This section covers the setup of the Cuckoo Sandbox environment used for sample 3. This environment not only differs from the hybrid environment in the way that it is set up, but once setup is complete, it is largely based on an automated analysis process.

Our setup of Cuckoo is largely based on the guide found here [20], which in turn is based on Cuckoo’s official documentation. Importantly, Cuckoo has been updated since the mentioned guide was made. Certain parts of it are therefore outdated or incomplete with regard to the current version of Cuckoo (2.0.7). For instance, Cuckoo does not perform network analysis by simply following the steps in the guide above, as Cuckoo’s storage structure was changed with version 2.0.0. From this version on, Cuckoo stores all of its files in the ’∼/.cuckoo’ directory, where the network utility *tcpdump* lacks write permission by default on Ubuntu [21]. For the network analysis to work, it is therefore necessary to install an additional utility that allows tcpdump to write to the new network dump location at ’∼/.cuckoo/storage/analyses/<task_id>/dump.pcap’:

```
$ sudo apt-get install apparmor-utils
$ sudo aa-disable /usr/sbin/tcpdump
```

Note also that after each restart of the host OS (Ubuntu in our case), it is necessary to run the commands below.

In the virtual environment (virtualenv) of the *cuckoo* user:

```
$ vmcloak-vboxnet0
```

On the default user:

```
$ sudo sysctl -w net.ipv4.conf.vboxnet0.forwarding=1
$ sudo sysctl -w net.ipv4.conf.wlp3s0.forwarding=1
```

In addition, all ’sudo iptables ...’ commmands will need to be re-run.

Alternatively, the commands can be added to a startup script or similar, which would mitigate the need of running these commands after every restart of the host OS.

Once Cuckoo is setup successfully, running the command ’cuckoo init’ should generate the following message:

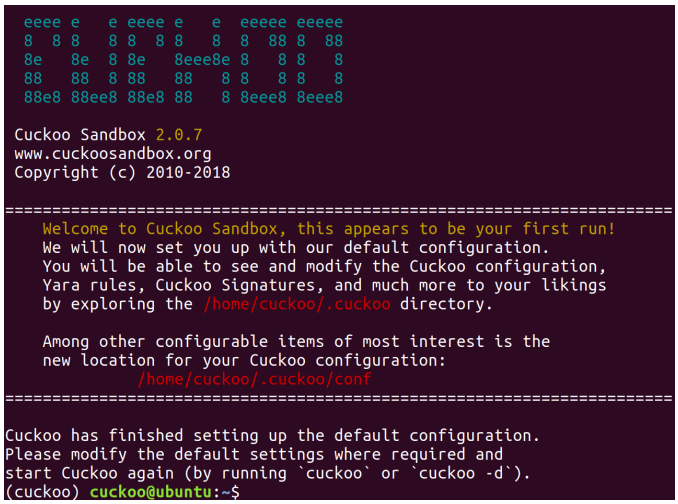


Fig. 2. Output from command ’cuckoo init’

VI. Analysis

- Practical analysis and showcase of tools
- Static and dynamic analysis
- Brief explanation of tools
- Screenshots/Snapshots
- Brief explanation of the malware type, history and behavior.

A. Sample 1 - 1855c5782eaa8b535345d71d18ff3a9f.exe

The first sample was 1855c5782eaa8b535345d71d18ff3a9f.exe [11]. This sample was retrieved from the malware sample exchange website MalwareBazaar [22].

1) Static Analysis: Before going into any tools we can perform the most basic of static analysis and that is the visual observation of the file itself. The file has no icon, it might even seem like it was painted over with a gray color.

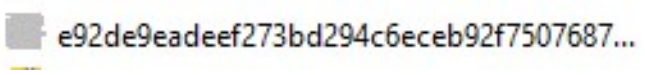


Fig. 3. Icon

Another thing is checking the properties, but here again, we get basically nothing as it is empty.

A lot of the time malware will try to disguise itself as something else, things like legitimate software or legitimate files like PDFs. This malware sample doesn't seem to have put any effort into that regard. Now we can actually go into some simple tools that will tell us if the file was packed or compressed. Using Exeinfo PE we get that the sample was unpacked so no packing was done and using PEStudio we can check the raw and virtual size of the file. If these two are very different in size that would indicate compression. Here we found that the sizes are equal which means no compression has been done.

Either using CFF Explorer or again PEStudio we can look at the original data of the file.

Property	Value
InternalName	Enjoyer
FileDescription	Veges
LegalCopyright	Copyright (C) 2022, Junior
OriginalFilenames	glitters
ProductsVersion	88.78.58.79
ProductName	Butilsken

Fig. 4. Original File Data

Here we can see the original file name "glitters" and then interestingly some sort of product name "Butilsken". Trying to find the product "Butilsken" on the internet only leads us to other malware that was found by malware analysis vendors. All of them use the same product name "Butilsken" and all of them originate from Ukraine, although the original file names are random and have no meaning. The file name "glitters" seems to have also been used on another malware.

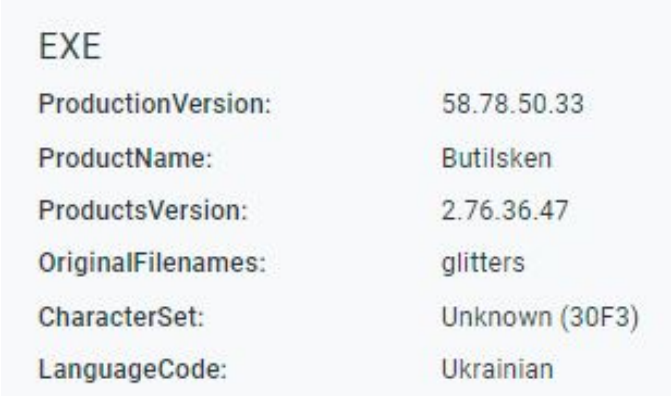


Fig. 5. Butilsken Information

The next step will be to focus on PEStudio which is one of the most useful tools for static analysis. It contains a window for libraries which shows what types of libraries the malware might use. Here only one was suspicious and flagged which was the WINHTTP.dll which is commonly used for HTTP and HTTPS requests which may suggest that the malware may be capable of network communication.

Now the next section will go over quite possibly the most useful one which is the strings. Here PEStudio basically sorts and turns all the information into readable strings. This is one of the most useful functions of PEstudio as it provides a lot of information about the file. Going over all the strings we notice that around 60% of them are just random numbers and letters which don't really have any meaning. Then we can notice some strings that sound like another language including a string that says "Translation". This makes us wonder if maybe these strings can be translated. We tried to translate some of them with no results except one which is "Mazucu nokuyicesu xevija" which Google Translate detected to be in the Zulu language which is used in Southern Africa and it translates to "It's a piece of cake". We're not sure if this has any relevancy since we have mentioned before that the country indicates to Ukraine, but it almost sounds like the creators of this malware making jokes saying creating this malware is a piece of cake or something similar.

Now what we think is the main function of this malware involves the hard drives. A lot of the strings point to hard drive functions. These include things like "SetVolumeLabel", "DeleteVolumeMountPoint", "GetVolumePathName", "RemoveDirectory".

The "DeleteVolumeMountPoint" suggests that this malware may try to delete the hard drive volume itself and that the malware may have capabilities to make changes to the boot sector. We remember the WINHTTP.dll that the malware

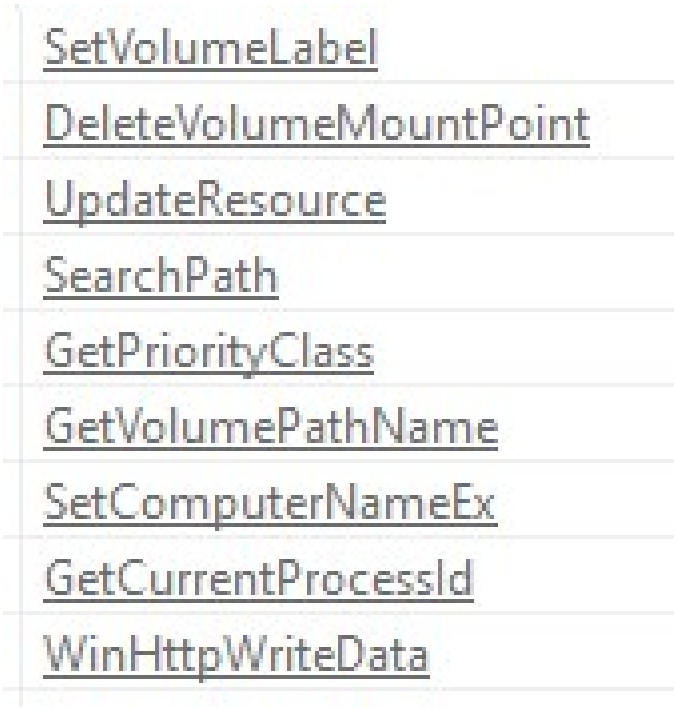


Fig. 6. Strings

had and it seems to only be used for one thing which is "WinHttpWriteData" which is used to request data to an http server. Can't really see where this leads without running a dynamic analysis. We can also see some reconnaissance functions to get current process, current process ID, current thread ID etc. There doesn't really seem to be anything else that isn't related to the hard drive. So we can safely say this malware targets the hard drives. Further investigation requires dynamic analysis which will be the next step since no more useful information can be acquired using static analysis.

2) Dynamic Analysis: The dynamic analysis of the sample 1855c5782eaa8b535345d71d18ff3a9f.exe was performed using two virtual machines: Windows 7 VM and REMnux VM. The hypervisor used for this was Oracle VirtualBox Version 7.0.12 r159484 (Qt5.15.3) [23] on an Ubuntu host system. Both of the virtual machines were configured to use a virtual network by attaching each machine to a host-only adapter (vboxnet0), enabling them to communicate with each other, but not with the Internet or the host machine.

The following procedure was used for performing dynamic analysis on the sample:

Pre-execution:

- Start Windows 7 VM Base Analysis Snapshot
- Run Regshot
  - Take '1st Shot'
- Run Process Monitor
- Run Process Explorer
- Create new Wireshark network capture on REMnux VM

Execution:

- Run 1855c5782eaa8b535345d71d18ff3a9f.exe
- Wait 60 seconds

Post-execution:

- Stop Wireshark network capture on REMnux VM
- Take '2nd Shot' using Regshot
- Compare results in Regshot
- Restore to Windows 7 VM Base Analysis

The tools used for the analysis of this sample were Process Explorer, Process Monitor, Wireshark and Regshot. This analysis will have a particular emphasis on the latter utility, due to its high efficiency and excellent compatibility with older operating systems such as Windows 7. Where the newest version of Regshot is compatible with Windows 7, the newest versions of Process Explorer and Process Monitor are not, meaning older versions of these two tools had to be dug up.

Note that the malware was run with both standard and elevated privileges, to ensure that it was given rights necessary to perform privileged actions. This did not appear to affect the results of the analysis in any way. Nonetheless, it should always be assumed that malware may behave in a different manner when executed normally and as an administrator.

We also make sure to terminate the process that was generated after running the executable, in case the malware is programmed to do something specific when terminated. This was done by right clicking the created process in Process Monitor and choosing "Kill Process Tree" – on one occasion before and on another occasion after taking the second shot with Regshot.

Interestingly, after running the malware, the first thing we notice is that it does not remove the file that was originally executed. This is typically implemented by malware authors to remove traces of the malware and avoid detection. Because this is not the case here, it could be an early indicator of malware that was written by a novice or a Trojan horse that wants the user to think they are dealing with a benign program.

Because we are dealing with dangerous software, there is still a good chance that the malware has spread itself onto other parts of the infected system, to ensure that it stays intact from there. To investigate this, we can use Regshot. This allows us to take one image/snapshot of the system before and one after running the malware, and then comparing the changes that were made during that interval. This can be a highly effective procedure, as it tells us what files, directories and registries were created, altered or deleted on the system between taking the two shots.

Comparing the 1st and 2nd snapshots of the systems registry and "C\:Windows;" directory reveals a total of 127 changes.

```
---------------------------------
Files [attributes?] modified: 16
---------------------------------
C:\Windows\Microsoft.NET\Framework\v2.0.50727\ngen_service.log
C:\Windows\Microsoft.NET\Framework64\v2.0.50727\ngen_service.log
C:\Windows\Prefetch\AUDIODG.EXE-BDFD3029.pf
C:\Windows\Prefetch\CMD.EXE-4A81B364.pf
C:\Windows\Prefetch\CONHOST.EXE-1F3E9D7E.pf
C:\Windows\Prefetch\CONSENT.EXE-531BD9EA.pf
C:\Windows\Prefetch\DLLHOST.EXE-766398D2.pf
C:\Windows\Prefetch\MSCORSVW.EXE-245ED79E.pf
C:\Windows\Prefetch\MSCORSVW.EXE-90526FAC.pf
C:\Windows\Prefetch\PING.EXE-7E94E73E.pf
C:\Windows\Prefetch\RUNDLL32.EXE-DE9673F9.pf
C:\Windows\Prefetch\SVCHOST.EXE-80F4A784.pf
C:\Windows\System32\config\SOFTWARE
C:\Windows\System32\config\SOFTWARE.LOG1
C:\Windows\System32\config\SYSTEM
C:\Windows\System32\config\SYSTEM.LOG1
```

Fig. 7. Files modified in snapshot comparison

Registered changes by Regshot:

- 2 registry keys deleted (-)
- 16 registry keys added (+)
- 4 registry value deleted (-)
- 45 registry values added (+)
- 27 registry values modified (%)
- 5 files added (+)
- 4 files deleted (-)
- 16 files modified (%)
- 2 folders added (+)
- 6 folders deleted (-)

The snapshot comparison shows that running the malware results in a file being created in the 'C\Windows\Prefetch' directory. This file has an identical name, but the different .pf extension. This could be seen as an effort by the malware to increase its own persistence, but most Windows Prefetch-related files are simply used by the OS to improve application start-up performance by loading application data into memory before it is demanded [24]. Because it represents a record of the first and last times an executable was executed (as well as where it was executed from and how many times it has been executed), this file would have been created immediately after running most other programs for the first time, as part of a natural and benevolent routine followed by the OS.

Interestingly, the comparison file also illustrates how several files in the Prefetch directory have been modified (as opposed to added), which indicate that the corresponding .exe-files were executed between the first and second snapshots. Whether or not this was caused by the malware is difficult to say, as most of these files are commonly used in the Windows OS.

Other changes reported by Regshot are mostly related to seemingly benign registry keys added, changed or deleted in 'HKLM\Software\Microsoft\Fusion\NativeImagesIndex' and files/folders changed in 'C:\Windows\assembly' and 'C:\Windows\Microsoft.NET' subfolders.

3) Summary: In summary this sample definitely looks suspicious and is most likely malicious. All the acquired information leads us to this assumption especially with the original file leading to previous instances of malware. It doesn't really try to disguise or hide itself. Our findings indicate that the main function of the malware is some sort of hard drive tampering. This is especially clear in static analysis with a lot of the strings pointing to hard drive functions like "DeleteVolumeMountPoint" and "GetVolumePathName" although sadly dynamic analysis doesn't really provide us with a lot more information than that. Our final conclusion points to this malware being some sort of boot virus or even a bootkit since it's possible it is trying to rewrite or infect the master boot record (where the operating system is located). This could also indicate ransomware, but we didn't encounter any encryption of files. After the analysis is done we can check the malware sample in VirusTotal and some vendors led to a malware family called "Pitou" which is part of a bootkit so our conclusion wasn't too far off.

B. Sample 2 - 0c5b40fbabd22d7b3717286e6e4df432.exe

The second sample we performed malware analysis on was 0c5b40fbabd22d7b3717286e6e4df432.exe [12]. This sample was retrieved from the malware sample exchange website MalwareBazaar, a website made with the intent of collecting and sharing malware samples [22].

Listed in Table III (see Malware Sample 2 in C. Methodology) is the general information about the listed sample 0c5b40fbabd22d7b3717286e6e4df432.exe fetched from MalwareBazaar, at the time of writing, the 11th of November 2023 [12].

1) Static Analysis: When working with static analysis, the goal will usually be to gather as much information as possible without running the malware. we can start by looking at the file we suspect to be malware and the first thing we see is that it looks like a windows cabinet self extractor. One would expect this file to present us with other files however when it is executed it just runs its malicious script instead. Some malware like to hide themselves in this way to trick both security systems and its users. we can notice that there is more than meets the eye by taking a look with other tools like Exeinfo PE to see if this file is packed or not. We do indeed find out that this is a cabinet file which contains compressed files and that it has been made in 2022. We will continue to unzip this file with 7zip and find the files hidden within. After unzipping the file a couple times we end up with 4 exe files that we will take a closer look at.

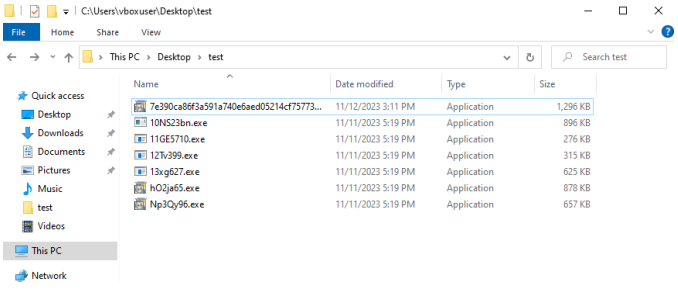


Fig. 8. exe files in folder

The next step will be to see which strings and library's the files has access to. By looking at strings in the context of static analysis, one can find out a considerable amount of useful information that will help with the dynamic analysis later on. Something to take notice of is that when we examine the original file and compare it with others, most of the libraries are only visible in the extracted files, not in the original one. we can also see that it uses the cabinet dll to decompress the files. In the picture bellow we will see an example of an operation this program runs where it takes "InternetConnectA", "InternetReadFile", "InternetGetCookieA", according to Microsoft documentation the MS functions "InternetConnectA", "InternetReadFile" will open and initialize an http or FTP session while "InternetGetCookieA" retrieves cookies from a specific url which is quite unexpected for a regular program (Microsoft, 2022). we will continue using PE studio which has sorted the information in a readable way that makes it quite easy to find the information you could consider suspicious. An example of this is that here we found that the file has access to and uses "WININET.dll", "ws2 32.dll". These are Windows dynamic-link Libraries that this file will use since the DLL is available for all processes on your computer. Ws2 32 is used for Winsock API which provides the capability of TCP / IP networking functions while WININET gives the machine access to HTTP AND FTP protocols which gives applications access to resources from the internet.

2) Dynamic Analysis: The dynamic analysis of the sample 0c5b40fbabd22d7b3717286e6e4df432.exe was performed using two virtual machines: Windows 10 VM and REMnux VM. The hypervisor used for this was VMware Fusion Professional Version 12.2.5 [25]. Both of the virtual machines were configured to use a virtual network (vmnet3) enabling them to communicate with each other, but not with the Internet or the host machine.

The following procedure was used for performing dynamic analysis on the sample:

*This was repeated three times**

Pre-execution:

- Start Windows 10 VM Base Analysis Snapshot
- Run Regshot
  - Take '1st Shot'
- Run Process Monitor
- Run Process Explorer
- Create new Wireshark network capture on REMnux VM

Execution:

- Run 0c5b40fbabd22d7b3717286e6e4df432.exe
- Wait 60 seconds

Post-execution:

- Stop Wireshark network capture on REMnux VM
- Take '2nd Shot' using Regshot
- Compare results in Regshot
- Open Autoruns, see if any new entries were added
- Restore to Windows 10 VM Base Analysis

The tools used for the analysis of 0c5b40fbabd22d7b3717 286e6e4df432.exe were Process Explorer, Regshot, Process Monitor, Wireshark, and Autoruns.

The first step in the analysis was to execute and open Process Explorer. Process Explorer is a tool part of Sysinternals that can be used to manage processes and their subprocesses [26]. For the analysis, we used this tool to get an overview of the sample and its child processes. In addition, it was used to terminate the process and to make sure it was not running anymore.

After opening Process Explorer, we ran Regshot and performed the "1st Shot" in order to take a snapshot of the current state of the system [27]. We chose to save logs in an HTML format and to take a snapshot of the directory "C\:Windows;" as well. By doing this we were able to take an initial snapshot of the registry and "C:\Windows;", before executing the sample.

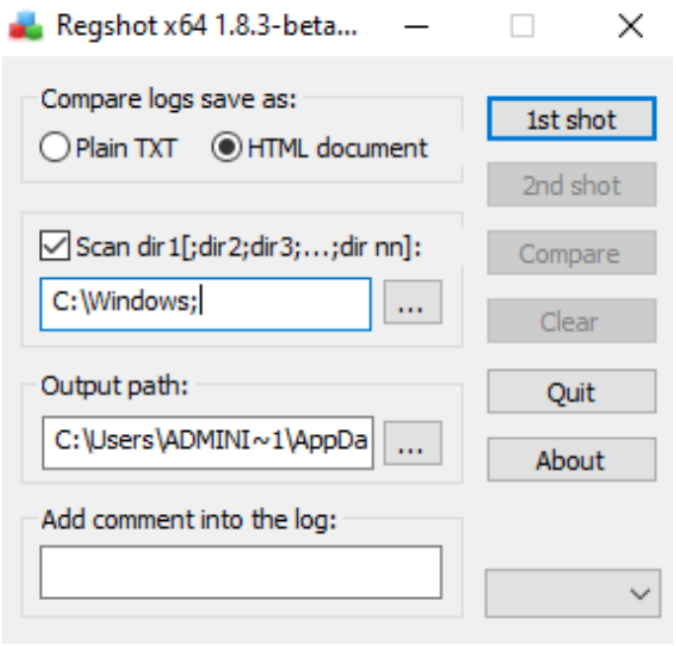


Fig. 9. Regshot 1st Shot

When the registry and directory snapshot were completed, we proceeded with executing the program Process Monitor. Process Monitor is a tool for monitoring Windows, collecting logs for the file system, registry, and process/thread activity [28]. For the analysis, we used Process Monitor to oversee the activity of the sample, looking at what processes it created and started.

After this, we started a new network capture in Wireshark on the REMnux VM in order to collect the network traffic

going between the Windows and REMnux virtual machines.

We then proceeded with executing the sample 0c5b40fbabd22d7b3717286e6e4df432.exe on the Windows virtual machine. The sample was allowed to run for 1 minute and 56 seconds on the virtual machine. This was the amount of time it took for the sample to perform all of its actions. This means the time it took for there to no longer be any visually observable changes in the system.

After terminating the sample 0c5b40fbabd22d7b3717286e6e4df432.exe using Process Explorer, we proceeded with stopping the Wireshark network capture, saving the Process Monitor logs and taking a 2nd system snapshot using Regshot.

Comparing the 1st and 2nd snapshots of the systems registry and "C\:Windows;" directory revealed a total of 217 changes. When quickly skimming over the changes, there does not seem to be any malicious changes performed by the sample.

Registered changes by Regshot:

- 2 registry keys deleted (-)
- 37 registry keys added (+)
- 1 registry value deleted (-)
- 61 registry values added (+)
- 67 registry values modified (%)
- 9 files added (+)
- 40 files modified (%)

Inspecting the autorun entries using the program Autoruns reveals no particularly suspicious entries indicating the sample attempting to establish persistence on the Windows virtual machine.

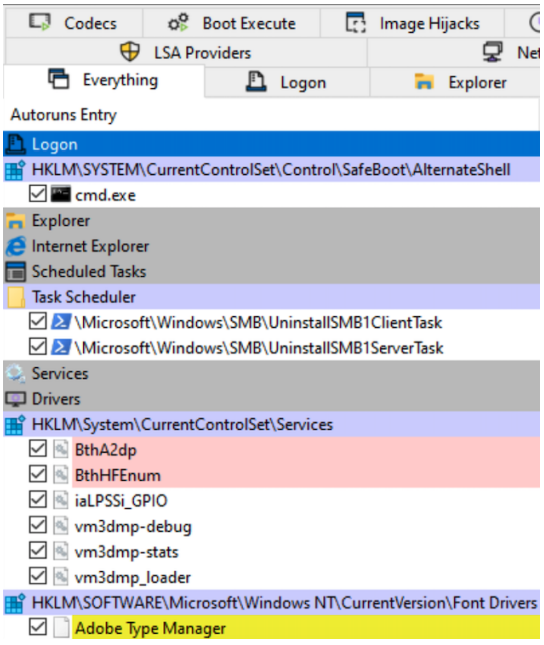


Fig. 10. Autoruns entries on the Windows VM

After terminating the sample, a total of eleven new tabs going to various websites were open in the default browser Microsoft Edge on the Windows virtual machine.

Websites opened:

- https://accounts.google.com
- https://facebook.com/login
- https://accounts.google.com
- https://store.steampowered.com/login/
- https://twitter.com/i/flow/login
- https://steamcommunity.com/openid/loginform/
- https://www.epicgames.com/id/login/
- https://www.paypal.com/signin
- https://www.youtube.com
- https://accounts.google.com

Looking at the process tree of the sample 0c5b40fbabd22d7b3717286e6e4df432.exe from the saved Process Monitor log file it becomes clear that the websites being opened are directly tied to the execution of the sample.

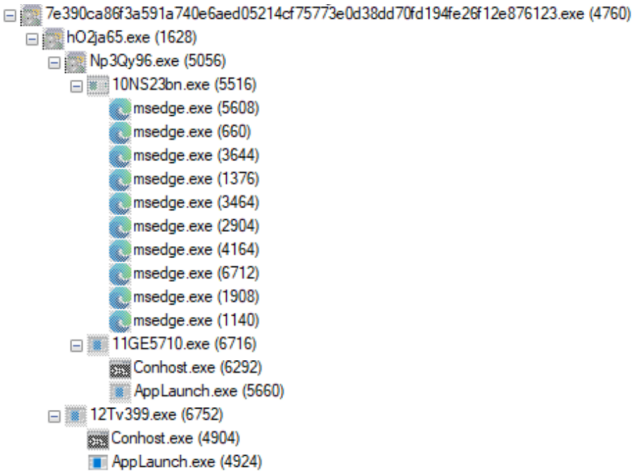


Fig. 11. Proccess tree for the sample (Process Monitor)

From the process tree of the sample, we can see that the sample created five subprocesses. These subprocesses opened msedge.exe (Microsoft Edge) ten times, and opened Conhost.exe and AppLaunch.exe two times. The fact that all of the websites opened are the login pages of popular services such as Facebook, Google, and PayPal is an indicator that something suspicious is going on.

The majority of the captured network traffic using Wireshark was TCP and HTTP traffic going back and forth between 192.168.46.128 (Windows VM) and 192.168.46.129 (REMnux VM). This could be explained by the internal Windows services reaching out to the Internet and the ten websites that were opened by the sample. Another finding was external connection attempts to the IPv4 address 5.42.92.43. This IPv4 address is tied to Stockholm, Sweden, and is owned by Altawk according to the RIPE database entry for the IPv4 address [29]. Searching for the domain associated with

Altawk in VirusTotal, altawk[.]net returns results indicating that the domain is associated with malicious activity [30].

Findings:

- 0c5b40fbabd22d7b3717286e6e4df432.exe creates five subprocesses
- The subprocesses open suspicious login websites to popular services
- Subprocesses execute conhost.exe and AppLaunch.exe
- The sample performed external connections towards the suspicious IPv4 address 5.42.92.43

3) Summary: From the static malware analysis of the sample, we gathered that the suspected file, initially appearing as a Windows cabinet self-extractor, behaves deceptively by running a malicious script. Further investigation using tools like Exeinfo PE revealed that the file is packed, and unpacking it exposes four executable files. Analysis of the strings of the executables indicates suspicious activities, such as unexpected operations like initiation of HTTP sessions. Tools such as PE Studio highlight the file's access to Windows dynamic-link libraries (DLLs), enabling communication with internet resources.

The dynamic analysis of the sample file included analysis of a network capture, analyzing system changes, and inspecting autorun entries. The analysis revealed 217 changes in the system, including registry key additions and deletions, file additions and modifications, and changes in autorun entries. However, no immediately suspicious changes were detected. The sample opened multiple tabs in the web browser Microsoft Edge, leading to login pages of popular services like Google, Facebook, and PayPal. Network traffic analysis showed external connection attempts to the IPv4 address 5.42.92.43 in Stockholm, Sweden, associated with malicious activity according to VirusTotal.

The sample has the functionality to retrieve browser cookies with the function "InternetGetCookieA". It also opens several Microsoft Edge web browser tabs to login pages of popular services. Lastly, the sample communicates with the IP address 5.42.92.43 which has a malicious reputation. All of the factors combined lead us to the conclusion that this sample is an info stealer that extracts passwords from a victim's browser and sends them to an external location.

C. Sample 3 - 12f4b96090bb7a1aa45768f68870ef90.exe

The third sample was 12f4b96090bb7a1aa45768f68870ef90 .exe [11]. This sample was retrieved from MalwareBazaar [22] and analyzed using Cuckoo Sandbox.

1) Static Analysis: Because Cuckoo's web interface provides a dedicated section/tab for statically analyzing a submitted sample, this portion of the analysis is primarily conducted within that specific section/tab (named "Static Analysis").

One of the first things we notice here is that the OriginalFilename property of the sample is set to 'StatusMessageResourceLib.dll'. This is already a clear indicator of a packed executable. This is supported by examining the different sections of the sample, most of which have highly varying virtual sizes and raw data sizes. In addition, there are three unusual section names, specifically '.vmp\xc2\xae?', where ? indicates 0, 1 and 2 for the three different sections. These also vary in size based on whether they are in memory or on disk.

Sections

Name	Virtual Address	Virtual Size	Size of Raw Data	Entropy
.text	0x00001000	0x0030e2c5	0x00000000	0.0
.rdata	0x00310000	0x0003c0aa	0x00000000	0.0
.data	0x0034d000	0x00012490	0x00000000	0.0
.pdata	0x00360000	0x00015690	0x00000000	0.0
_RDATA	0x00376000	0x000000fc	0x00000000	0.0
.vmp\xc2\xae0	0x00377000	0x002590fc	0x00000000	0.0
.vmp\xc2\xae1	0x005d1000	0x00000c38	0x00000e00	0.135595803147
.vmp\xc2\xae2	0x005d2000	0x004ca2ec	0x004ca400	7.97119978288
.reloc	0x00a9d000	0x0000157c	0x00001600	5.42132125373
.rsrc	0x00a9f000	0x00066ee7	0x00067000	5.68650390139

Fig. 12. Sections identified by Cuckoo

Cuckoo further reveals that the sample contains some potentially malicious imports. One of these is the ShellExecuteA function from the library SHELL32.dll, which can potentially be used by an attacker to execute commands and open other files or programs [31]. The sample also notably imports GetSystemTimeAsFileTime from KERNEL32.dll, which represents a commonly used function in malware [32][33].

While a lot of malware will reveal key information among their strings, this particular sample appears to obfuscate this well. Cuckoo does provide a long list of strings, but few (if any) of these reveal any meaningful information at first glance.

2) Dynamic Analysis: When moving on to the dynamic analysis, we immediately notice that Cuckoo lists several events of high concern among the signatures in the summary section (figure 13). Many of these events are related to the sample attempting to disable Windows Security features and modifying Windows Defender policies, through the use of the Windows registry.

To investigate these events in more detail, Cuckoo has a dedicated section for looking into the behavioral analysis of the sample. Here, we can dig further into the information of interest by clicking on the red 'registry' button, which reveals a large amount of registry activity performed by the malware. Many of these are directly related to Group Policy Objects of Windows Defender, with the sample attempting to do everything from retrieving registry information (RegOpenKeyExA), to editing key values (RegSetValueExA), and deleting key entries (RegDeleteKeyW), all related to Windows Defender. One notable example is that the malware uses RegSetValueExA to change the value of DisableRealtimeMonitoring from 0 to 1 (figure 14), which would effectively turn off Windows Defender's real-time protection from known malware [34].

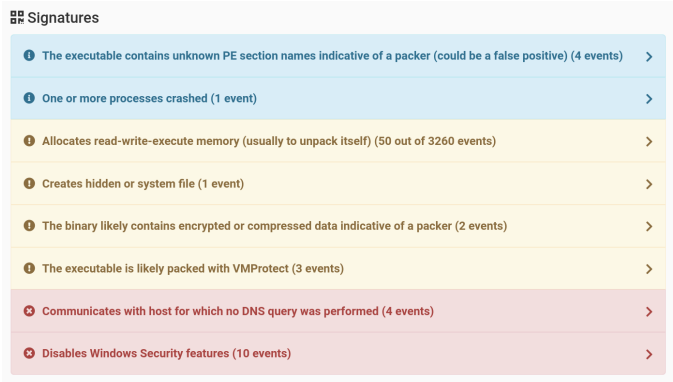


Fig. 13. Signatures identified by Cuckoo

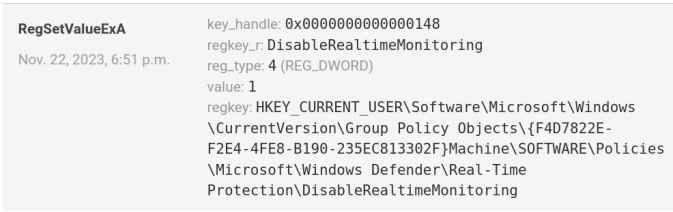


Fig. 14. Value of DisableRealTimeMonitoring changed to 1 (true)

Furthermore, in the network analysis section of Cuckoo's web interface, it becomes evident that the malware attempts to communicate with several suspicious IP addresses (hosts). In addition, we see some interesting HTTP(S) and DNS traffic. While none of the domains listed here are malicious, it is reasonable to assume that this is used by the malware in order to determine whether it has internet connectivity.

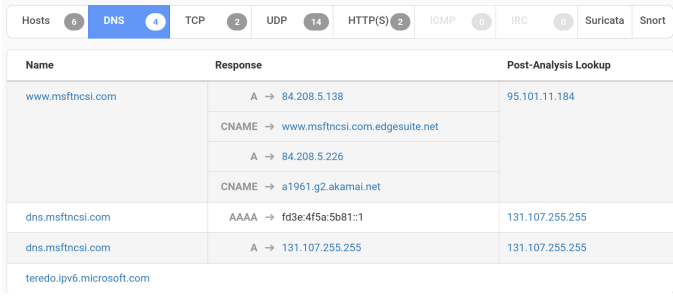


Fig. 15. DNS traffic identified by Cuckoo

3) Summary: In summary, it is highly likely that this is packed malware that unpacks itself by allocating memory through the use of various memory-related functions. After doing this, the malware tries its best to disable as much security functionality as possible in Windows Defender. Changing of registry keys such as 'DisableAntiSpyware' and 'DisableRealtimeProtection' from *false* to *true* is indicative of potential spyware. This is supported by suspicious network activity, which could be used by the malware in an effort to send logged data to other controlled hosts.

VII. Conclusion

At the start of the project, we set out to use Cuckoo Sandbox to perform automated analysis on malware samples for the project. Configuring and installing Cuckoo was more difficult than we initially thought and we ended up largely using a more traditional manual sandbox environment instead. One of the group members ended up being able to use and install the Cuckoo sandbox which they used to do static and dynamic analysis on the third malware sample. Initially, we had emphasized using automated analysis, but as we progressed with the project, it was necessary to adjust our course for the reasons noted. As a result, the analysis in the final project skews towards the importance of synergy between static and dynamic malware analysis.

During the project, we learned the importance of combining both static- and dynamic malware analysis to get a complete picture of the capabilities and behaviors of a program. For instance, static analysis alone could lead us to the conclusion that the program is a video game using functions to fetch the users' keyboard inputs. Using this information, you would be able to know the capabilities of the program and potentially get a few ideas about the intentions behind it. Combining this with dynamic analysis it could reveal that the assumed game is in reality a keylogger collecting the users' keystrokes.

While all group members have had some prior experience with malware analysis, we feel this project has enabled us to become more comfortable and knowledgeable on this subject. In particular, the project has allowed us to extend our area of expertise within troubleshooting and automated analysis, which may well prove useful in future malware-related endeavors.

VIII. References

[1] Red Hat. "What is a virtual machine (VM)?" In: (2022). URL: https://www.redhat.com/en/topics/virtualization/what-is-a-virtual-machine.

[2] Neil Fox. "Cuckoo Sandbox Overview". In: (2023). URL: https://www.varonis.com/blog/cuckoo-sandbox.

[3] Kurt Baker. "MALWARE ANALYSIS". In: (2023). URL: https://www.crowdstrike.com/cybersecurity-101/malware/malware-analysis/.

[4] VMRay. "How Dynamic Analysis Works". In: (2023). URL: https://www.vmray.com/glossary/dynamic-analysis/.

[5] Microsoft. "GetKeyState function (winuser.h)". In: (n.d). URL: https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-getkeystate.

[6] National Institute of Standards and Technology. "key logger". In: (n.d). URL: https://csrc.nist.gov/glossary/term/key_logger.

[7] Ben Lutkevich. "malware". In: (n.d). URL: https://www.techtarget.com/searchsecurity/definition/malware.

[8] proofpoint. "What Are Indicators of Compromise?" In: (n.d). URL: https://www.proofpoint.com/us/threat-reference/indicators-compromise.

[9] Bart Lenaerts-Bergmans. “WHAT ARE COMMAND AND CONTROL (CC) ATTACKS?” In: (2023). URL: https://www.crowdstrike.com/cybersecurity-101/cyberattacks/command-and-control/.

[10] Cuckoo Foundation. “Per-Analysis Network Routing”. In: (2020). URL: https://cuckoo.readthedocs.io/en/latest/installation/host/routing/.

[11] Malware Bazaar. “Malware Sample 1”. In: (2023). URL: https://bazaar.abuse.ch/sample/e92de9eadeef273bd294c6eceb92f750768766a79c215843e948f37b95bb6723/.

[12] Malware Bazaar. “Malware Sample 2”. In: (2023). URL: https://bazaar.abuse.ch/sample/7e390ca86f3a591a740e6aed05214cf75773e0d38dd70fd194fe26f12e876123/.

[13] Malware Bazaar. “Malware Sample 3”. In: (2023). URL: https://bazaar.abuse.ch/sample/ff63976d5cc3f2965ef707e4073440ca0fd9d976a02ccc766bf7a4d9af9144e9/.

[14] Ricardo van Zutphen. “Cuckoo Sandbox 2.0.7”. In: (2019). URL: https://cuckoosandbox.org/blog/207-interim-release.

[15] Windows Central. “Setting up Windows 11 without Microsoft Account”. In: (2023). URL: https://www.windowscentral.com/how-set-windows-11-without-microsoft-account.

[16] João Carrasqueira. “Can I run Windows 11 without a Microsoft account?” In: (2023). URL: https://www.xda-developers.com/windows-11-microsoft-local-account/.

[17] Fortinet. “What Is Malware Analysis?” In: (n.d). URL: https://labs.vipre.com/dga-malware-usage-and-known-infections-part-2/.

[18] University of Maryland Baltimore County. “Inetsim/Virtual Lab installation and configuration guide:” in: (n.d). URL: https://redirect.cs.umbc.edu/courses/undergraduate/CMSC491malware/installGuide.pdf.

[19] NTNU. “Analysis Platform”. In: (2022).

[20] Alwin Peppels and Ricardo van Zutphen. “Cuckoo Sandbox Setup for People in a Hurry”. In: (2019). URL: https://hatching.io/blog/cuckoo-sandbox-setup/.

[21] Cuckoo Foundation. “Permission denied for tcpdump”. In: (2019). URL: https://cuckoo.sh/docs/faq/index.html#permission-denied-for-tcpdump.

[22] Malware Bazaar. “About”. In: (n.d). URL: https://bazaar.abuse.ch/about/.

[23] Oracle. “Download VirtualBox for Linux Hosts”. In: (2023). URL: https://www.virtualbox.org/wiki/Linux_Downloads.

[24] Joachim Metz. “Windows Prefetch File (PF) format”. In: (2023). URL: https://github.com/libyal/libscca/blob/main/documentation/Windows%20Prefetch%20File%20(PF)%20format.asciidoc.

[25] VMWare. “VMware Fusion”. In: (n.d). URL: https://www.vmware.com/products/fusion.html.

[26] Mark Russinovich. “Process Explorer v17.05”. In: (2023). URL: https://learn.microsoft.com/en-us/sysinternals/downloads/process-explorer.

[27] Seabreg. “Regshot”. In: (2019). URL: https://github.com/Seabreg/Regshot.

[28] Mark Russinovich. “Process Monitor v3.96”. In: (2023). URL: https://learn.microsoft.com/en-us/sysinternals/downloads/procmon.

[29] RIPE Database. “RIPE Database. 5.42.92.43,” in: (n.d). URL: https://www.virustotal.com/gui/domain.dltawk.net.

[30] VirusTotal. “altawk[.]net”. In: (n.d). URL: https://www.virustotal.com/gui/domain.dltawk.net.

[31] Microsoft. “ShellExecuteA function (shellapi.h)”. In: (2023). URL: https://learn.microsoft.com/en-us/windows/win32/api/shellapi/nf-shellapi-shellexecutea.

[32] Positive Technologies. “Sandbox detection and evasion techniques. How malware has evolved over the last 10 years”. In: (2021). URL: https://www.ptsecurity.com/ww-en.dnalytics/antisandbox-techniques/.

[33] Vipre Security Group. “DGA Malware-Usage and Known Infections (Part 2)”. In: (2017). URL: https://labs.vipre.com/dga-malware-usage-and-known-infections-part-2/.

[34] Group Policy Administrative Templates Catalog. “Turn off real-time protection”. In: (n.d). URL: https://admx.help/?Category=Windows_10_2016&Policy=Microsoft.Policies.WindowsDefender::DisableRealtimeMonitoring.